

HORT 7.0 — Supervised Learning Final Exercise

Traitz Tech Hort 7.0 Internship Program

Assignment Overview

You will apply everything you've learned so far — **Linear Regression, Logistic Regression, Decision Trees, KNN, and Classification Metrics** — on a real medical dataset: the **Heart Disease dataset**.

Goal: Build a model that predicts whether a patient has heart disease, based on their medical measurements.

This is the same style of workflow used in real industry projects: explore the data first, understand it, then model it — never jump straight to modeling blind.

Remember you have the internet before your internship supervisor.

Submission Deadline

Friday 28th August, 10:00 AM — Traitz Tech Office

Late submissions will not be reviewed. Submit as a .ipynb (Jupyter Notebook) or .py file with clear section comments.

The Dataset

Heart Disease Dataset (Kaggle)

Column	Meaning
age	Age in years
sex	1 = male, 0 = female
cp	Chest pain type (0-3)
trestbps	Resting blood pressure
chol	Serum cholesterol (mg/dl)
fbs	Fasting blood sugar > 120 mg/dl (1 = true, 0 = false)
restecg	Resting electrocardiogram results (0-2)

thalach	Maximum heart rate achieved
exang	Exercise induced angina (1 = yes, 0 = no)
oldpeak	ST depression induced by exercise
slope	Slope of peak exercise ST segment
ca	Number of major vessels colored by flourosopy (0-3)
thal	Thalassemia type
target	0 = no disease, 1 = has disease (this is what you're predicting)

Why this dataset? It's a **classification problem** — exactly what Logistic Regression, Decision Trees, and KNN were built for. It also mirrors the real-world Traitz use case of building predictive tools for health/service applications in the African context.

What You Must Do (Step by Step)

Part 1 — Exploratory Data Analysis (EDA)

Why: You must understand your data before you can trust any model built on it.

1. Load the dataset with pandas and print:
 - `data.shape` — how many rows/columns
 - `data.head()` — first few rows
 - `data.info()` and `data.dtypes` — check data types
 - `data.isnull().sum()` — check for missing values
2. Check the **target balance**:
 - Use `data['target'].value_counts()`
 - **Why this matters:** if one class massively outnumbered the other, accuracy alone becomes a misleading metric (remember the dog/cat example from the metrics guide)
3. Basic statistics:
 - Run `data.describe()` on the numerical columns (age, chol, trestbps, thalach, oldpeak)

- Note the min, max, and mean of at least 3 features in your write-up

4. **One simple visualization** (pick at least one):

- A bar chart of target counts
- A histogram of age
- A scatterplot of age vs chol, colored by target

Guide: use matplotlib or seaborn — a few lines of code is enough. You don't need to reproduce the full notebook sample; just show you understand your data.

Part 2 — Prepare the Data

Why: Models only understand numbers, and need clean, split data to train fairly.

1. Confirm all columns are numeric (this dataset already comes pre-encoded, so no need for extra encoding)
2. Split the data:
 - X = all feature columns (everything except target)
 - y = target
3. Use `train_test_split` — 75% train, 25% test (`test_size=0.25`, `random_state=42`)

Guide: always set `random_state` — this ensures your results are reproducible when you re-run the notebook.

Part 3 — Build Three Models

You must train and evaluate **all three** of the following on this dataset:

1. Logistic Regression

```
from sklearn.linear_model import LogisticRegression
```

2. Decision Tree Classifier

```
from sklearn.tree import DecisionTreeClassifier
```

3. K-Nearest Neighbors (KNN)

```
from sklearn.neighbors import KNeighborsClassifier
```

Guide:

- *For KNN — don't forget to scale your features first using StandardScaler, since KNN relies on distance and these features have very different ranges (age is 20-80, cholesterol is 100-500+)*
- *For Decision Tree — try max_depth=4 to avoid an overly complex tree on your first attempt*
- *For KNN — start with n_neighbors=5*

For **each model**, you must:

1. Train it (model.fit(X_train, y_train))
 2. Predict on the test set (model.predict(X_test))
 3. Print the predictions vs actual values for the first 10 test rows
-

Part 4 — Evaluate Each Model

For **each of your three models**, calculate and print:

1. **Accuracy**
2. **Confusion Matrix**
3. **Precision**
4. **Recall**
5. **F1 Score**

```
from sklearn.metrics import accuracy_score, confusion_matrix,  
precision_score, recall_score, f1_score
```

Guide: Build one small table (a printed DataFrame is fine) comparing all three models side-by-side on these five metrics — this makes comparison much easier than scattered print statements.

Part 5 — Written Analysis (Short — half a page max)

Answer these questions directly in your notebook using markdown cells or comments:

1. **Which model performed best overall?** Justify using your metrics table, not just accuracy.

2. **Why might accuracy alone be misleading here?** Check your target balance from Part 1 to support your answer.
 3. **In this medical context, which metric matters more — Precision or Recall? Why?** (*Hint: think about what happens if the model tells a sick patient they're healthy, vs telling a healthy patient they're sick*)
 4. **For KNN specifically:** try 2–3 different values of K (e.g., 3, 5, 9) and report how accuracy changes. What K would you recommend and why?
 5. **One thing you noticed while exploring the data** (Part 1) that surprised you or stood out.
-

✓ Submission Checklist

Before submitting, confirm you have:

- ☐ EDA section with shape, missing values check, target balance, and at least 1 chart
 - ☐ Data split into train/test correctly
 - ☐ Logistic Regression trained and evaluated
 - ☐ Decision Tree trained and evaluated
 - ☐ KNN trained and evaluated (with scaling applied)
 - ☐ A comparison table of Accuracy, Precision, Recall, F1 for all three models
 - ☐ Written answers to all 5 analysis questions
 - ☐ Code is commented and organized into clear sections (Part 1, Part 2, etc.)
-

💡 Notes Before You Start

- **Do not just copy code blindly.** If a line doesn't make sense, that's a signal to slow down and understand it — you'll be asked to explain your notebook during review.
- **Small dataset, real caution:** this dataset only has ~300 rows. Keep this in mind when interpreting your results — don't overstate

confidence in your findings, mention this limitation explicitly in your analysis.